

データ構造 グラフ理論 連結性 Union-Find アッカーマン関数 逆アッカーマン関数

Union-Find の計算量上界

1. 概要

本講座では、Union-Find の計算量が $O(N + Q\alpha(N))$ であるという事実を証明します。

この事実は競技プログラミングユーザーの多くが知っている非常に有名なものだと思いますが、Union-Find の基礎的な動作原理や、計算量 $O(N + Q\log N)$ であることの理解に比べて非常に難しくなるため、多くの解説でも省略されており、証明を学んだことがないユーザーもかなり多いと思います。ぜひこの機会に学んでみてください。

2. 前提知識

Union-Find のアルゴリズムの基本的な仕組みを理解していることが必要です。

特に、**path-compression** および、**union by size** による実装を仮定します。本講座内でもごく簡単にこれらを復習しています。

3. 主定理

3.1. 逆アッカーマン関数

本講座では、アッカーマン関数を定義することなく、直接逆アッカーマン関数を定義して議論します。

なおここで扱う定義は アッカーマン関数、逆アッカーマン関数 で扱っているものと同一ですが、本講座の議論では アッカーマン関数、逆アッカーマン関数 の内容は仮定しません。

非負整数 k と正整数 N に対して、 $\alpha_k(N)$ を次の式によって定義する。

$$\begin{aligned}\alpha_0(N) &= \left\lceil \frac{N}{2} \right\rceil, \\ \alpha_k(1) &= 0, \\ \alpha_{k+1}(N) &= 1 + \alpha_{k+1}(\alpha_k(N)) \quad (k \geq 0, N \geq 2).\end{aligned}$$

定義式は、次のように書くこともできます。

$$\alpha_{k+1}(N) = \min \left\{ n \mid \alpha_k^{(n)}(N) = 1 \right\}.$$

つまり $\alpha_{k+1}(N)$ は、 N に対して α_k をとる操作を反復したときに 1 に到達するまでの操作回数です。

小さな k, N に対する値は以下ようになります。

N	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
α_0	1	1	2	2	3	3	4	4	5	5	6	6	7	7	8	8	9	9
α_1	0	1	2	2	3	3	3	3	4	4	4	4	4	4	4	4	5	5
α_2	0	1	2	2	3	3	3	3	3	3	3	3	3	3	3	3	4	4
α_3	0	1	2	2	3	3	3	3	3	3	3	3	3	3	3	3	3	3

【定義 2】

正整数 N に対して、 $\alpha(N)$ を次の式によって定義する。

$$\alpha(N) = \min \{ k \mid \alpha_k(N) \leq 3 \}.$$

α を**逆アッカーマン関数** (inverse Ackermann function) という。

3.2. Union-Find で用いるアルゴリズム

Union-Find は、無向グラフに対する辺の追加、および、頂点の属する連結成分に関する情報の取得を行うデータ構造でした。特に、各連結成分を根付き木として管理し、根を連結成分の代表元として扱います。

Union-Find のメソッドとして、主に次の 2 つを考えます。

- `find(v)` : v の属する連結成分の代表元を返す.
- `unite(u, v)` : u, v の間に無向辺を追加する.

`find` については、本講座では **path-compression** と呼ばれる経路圧縮手法を採用します。つまり、以下のような実装となります。

```
int find(int v) {
    if (parent[v] == -1) return v; // v は根
    int r = find(parent[v]);
    parent[v] = r; // v の親を r に変更 (path-compression)
    return r;
}
```

`unite` については、**union by size** を採用します。（なお、union by rank という方法でも本講座の証明はほぼそのまま適用できます。）次のような実装が考えられます。

```
void unite(int u, int v) {
    u = find(u), v = find(v);
    if (u == v) return;
    if (size[u] < size[v]) swap(u, v); // union by size
    parent[v] = u, size[u] += size[v];
}
```

しかし、`unite` の呼び出し 1 回につき 2 回の `find` が行われることが議論を複雑化するため、本講座では `unite` 中に `find` を行うことは考えず、`unite` は連結成分の代表元同士についてのみ呼び出されることとします。

```
// u, v が異なる成分の根であることを前提とする
void unite(int u, int v) {
    assert(u != v && parent[u] == -1 && parent[v] == -1);
    if (size[u] < size[v]) swap(u, v);
    parent[v] = u, size[u] += size[v];
}
```

サンプル実装

- <https://judge.yosupo.jp/submission/351619> (Library Checker Unionfind)

3.3. 主定理

次の定理が目標です.

【定理 3】

頂点数 N の Union-Find に対し, `find`, `unite` のクエリを合計 Q 回行うとき, 計算量は全体で $O(N + Q\alpha(N))$ 時間である.

なお 3.2 の実装において, `unite` の計算量は $O(1)$ なので, `find` の計算量のみが問題です. さらに `find` では, v から根までのパスのうち親が変更されないものの個数は $O(1)$ なので, `find` によってある頂点の親が変更される回数が論点だということになります.

4. $h(v)$

Union-Find で管理する根付き森を F とします. これはもちろん, クエリの進行によって変化します.

途中でどのように `find` が行われても, これは `unite(u, v)` において u, v のどちらが根となるかの判定には一切影響しないことに注意します. そこで, 「クエリのうちで `unite` のみを行うとした場合」に得られる最終的な森」を F_0 とします. F_0 は最終的な森を指しており, これはクエリの進行によって**変化しない**ことに注意してください.

計算量解析において, 各頂点の F_0 における高さが主要な道具となります.

【定義 4】

頂点 v について, v の F_0 における高さを $h(v)$ と書く.

$h(v)$ という記号は主に F の頂点にも用いますが, 指すのは F_0 における高さであることに注意してください. 特に, $h(v)$ はクエリの進行によって**変化しません**.

主定理の証明において用いられる $h(v)$ の性質は、次の命題のもののみです。

【補題 5】

1. $h(v)$ は 0 以上 $N - 1$ 以下の非負整数である。
2. F においてどの時点においても、 u の子が v であるとき $h(u) > h(v)$ が成り立つ。
3. 任意の非負整数 k について、 $h(v) = k$ を満たす v の個数は $N/2^k$ 以下である。

1 は明らかです。

Union-Find のアルゴリズムを考えると、 F における v の親は常に F_0 における v の祖先であることが分かります。このことから 2 が成り立つことが分かります。

3 を示します。 $\text{size}_{F_0}(v)$ を、 F_0 における部分木 v の頂点数とします。 F_0 が union by size で作られた森であることから、 w が F_0 における v の子であるとき $\text{size}_{F_0}(v) \geq 2\text{size}_{F_0}(w)$ が成り立ちます。このことから $h(v) = k$ であるとき $\text{size}_{F_0}(v) \geq 2^k$ が成り立ちます。このことと高さ k の頂点の部分木同士は交わらないことから、 $h(v) = k$ となる頂点の個数は $N/2^k$ 以下です。 ■

5. 計算量解析

計算量解析においては、`find` によって親が変更される回数を調べればよいのでした。この回数を以下**コスト**と呼ぶことにします。

v の親が p から q に変更されるとします。計算量解析では、コストを組 $(h(v), h(p))$ によって分類して調べます。ひとつの v に対し、「 v の親に対する $h(-)$ の値」は親の変更によって増加することに注意してください。

議論を簡単にするため、次を自明ケースとして処理しておきます。

- $h(v) \leq 4$ の場合：そのような v は `find` 操作あたり $O(1)$ 個なので、 $O(Q)$ である。
- $h(p) \leq h(v) + 1$ の場合：そのようなコストは v ごとに 1 回以下なので、全体で $O(N)$ である。

以下, $5 \leq h(v) \leq h(v) + 2 \leq h(p)$ となる v に対するコストについて考えます. 以下, v, p, q と書けば特に断らずともこのようなもの考えることとします.

5.1. コストの階層への分類

$K = \alpha(N)$ とします. 特に $\alpha_K(N) \leq 3$ が成り立ちます.

α_k の定義から, 非負整数 k と $x_1, x_2 \geq 2$ に対して $\alpha_k(x_1) = \alpha_k(x_2)$ ならば $\alpha_{k+1}(x_1) = \alpha_{k+1}(x_2)$ が成り立ちます. したがって, $\alpha_k(h(v)) = \alpha_k(h(p))$ ならば $\alpha_{k+1}(h(v)) = \alpha_{k+1}(h(p))$ が成り立ちます.

$\alpha_K(N) \leq 3$ から, $5 \leq h(v) \leq h(p)$ のとき, $\alpha_K(h(v)) = \alpha_K(h(p))$ が成り立ちます. そこで $\alpha_k(h(v)) = \alpha_k(h(p))$ となる k のうち最小のものをとり, このコストを**階層 k のコスト**と呼ぶことにします. また $h(v) + 2 \leq h(p)$ としていることから $1 \leq k$ が成り立ちます. k の最小性より $\alpha_{k-1}(h(v)) < \alpha_{k-1}(h(p))$ が成り立ちます.

N	$\dots$	5	6	7	8	9	10	11	12	13	14	15	16	17	18	
α_0	$\dots$	3		4		5		6		7		8		9		
α_1	$\dots$	3				4									5	
α_2	$\dots$	3												4		
α_3	$\dots$	3														

$(h(v), h(p))$	k
(10, 15)	1
(3, 10)	2
(3, 18)	3

5.2. 階層ごとのコスト

【定理 3】 の証明のためには, 次を示せば十分です.

【補題 6】

各 $1 \leq k \leq K$ に対して、階層 k でのコストは $O(N + Q)$ である。

以下 k を固定します。コストをさらに 2 種に分類して数えます。

‘find’ によって v の親が p から q へ変更されるとします。

タイプ 1

$\alpha_{k-1}(h(p)) = \alpha_{k-1}(h(q))$ となるものについて考えます。

このような (v, p, q) は、‘find’ 1 回あたり 1 組しかありません。2 つの組 (v_1, p_1, q) および (v_2, p_2, q) があつたとして $h(v_1) < h(v_2)$ であるとする、 v_1, p_1, v_2, p_2 はこの順に同じパス上に並びます（ただし $p_1 = v_2$ であることはありえます）。よって

$$\alpha_{k-1}(h(v_1)) < \alpha_{k-1}(h(p_1)) \leq \alpha_{k-1}(h(v_2)) < \alpha_{k-1}(h(p_2))$$

です。特に $\alpha_{k-1}(h(p_1)) < \alpha_{k-1}(h(p_2))$ なので、これらがともに $\alpha_{k-1}(h(q))$ と等しいということはありません。

したがってこのタイプの (v, p, q) に対するコストは ‘find’ 操作あたり 1 回以下しかなく、 Q クエリを通して $O(Q)$ となります。

タイプ 2

$\alpha_{k-1}(h(p)) < \alpha_{k-1}(h(q))$ となるもの。

このようなコストを v を固定して調べます。 v に対して階層 k でこのようなコストが x 生じたとします ($x \geq 1$)。 $x - 1$ 回目のコストが生じた後の v の親を r とすると、

- $\alpha_k(h(v)) = \alpha_k(h(r))$ である (x 回目のコストが階層 k で生じるため)。
- $x \leq \alpha_{k-1}(h(r))$ である (コストが生じるたびに v の親の $\alpha_{k-1}(h(-))$ は増加するため)。

が成り立ちます。 $t = \alpha_k(h(v))$ とすると、 $t = \alpha_k(h(r))$ です。よって

$$2 \leq \alpha_{k-1}^{(t-1)}(h(v)), \quad 1 = \alpha_{k-1}^{(t)}(h(r))$$

です。また、 $x \leq \alpha_{k-1}(h(r))$ より

$$\alpha_{k-1}^{(t-1)}(x) \leq \alpha_{k-1}^{(t)}(h(r)) = 1 < 2 = \alpha_{k-1}^{(t-1)}(h(v))$$

となります。したがって $x \leq h(v)$ となります。

特にこのタイプのコストは各頂点 v ごとに、 $h(v)$ 以下です。その v にわたる総和は、【補題 5】より

$$\sum_{i=5}^{\infty} \frac{N}{2^i} \cdot i$$

以下となります。これは $O(N)$ です。以上により 【補題 6】 が示されました。 ■

5.3. 【定理 3】 の証明

階層ごとのコストを足し合わせることで、Union-Find の計算量は $O((N + Q)\alpha(N))$ であることが示されたことになります。

$N \leq Q$ ならばこれは $O(N + Q\alpha(N))$ なのでよいです。また $Q \leq N$ のときには、1 度以上 `find`、`unite` の対象になる頂点は $O(Q)$ 個しかないため、それらの頂点のみを対象としてこれまでの議論を行えば $N = O(Q)$ の場合に帰着できて、やはり計算量は $O(N + Q\alpha(N))$ であるということが出来ます。 ■

6. 関連問題

1. <https://judge.yosupo.jp/problem/unionfind>
2. https://atcoder.jp/contests/toyota2023spring-final/tasks/toyota2023spring_final_g

本講座の Union-Find 計算量解析を問う問題は、筆者は見たことがありません。

2 は本講座の内容に反して、**高速化の工夫を全くしない場合の** Union-Find の計算量について考える問題です。本講座の証明とは直接関係はありませんが、関連問題として紹介しておきます。

7. まとめ

本講座では、Union-Find の計算量が $O(N + Q\alpha(N))$ であるという事実を証明しました。

8. 参考文献

1. Robert Endre Tarjan. Efficiency of a Good But Not Linear Set Union Algorithm. Journal of the ACM. Vol. 22. No. 2. pp. 215–225. 1975. DOI: 10.1145/321879.321884. <https://doi.org/10.1145/321879.321884>.
2. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. アルゴリズムイントロダクション 第3版 総合版. 翻訳: 浅野 哲夫, 岩野 和生, 梅尾 博司, 山下 雅史, 和田 幸一. 近代科学社. 2013. ISBN: 9784764904088. https://www.kindaiakagaku.co.jp/book_list/detail/9784764904088/.
3. Raimund Seidel. Top-Down Analysis of Path Compression: Deriving the Inverse-Ackermann Bound Naturally (and Easily). Proceedings of Algorithm Theory – SWAT 2006 (SWAT 2006). Lecture Notes in Computer Science. Vol. 4059. 2006. DOI: 10.1007/11785293_1. https://doi.org/10.1007/11785293_1.
4. -is-this-fft-. [Tutorial] Proving the inverse Ackermann complexity of Union-Find. Codeforces Blog. <https://codeforces.com/blog/entry/98275>. (閲覧日: 2026-03-04).
5. kopricky. Union Find の計算量の話. Qiita. <https://qiita.com/kopricky/items/3e5847ab1451fe990367>. (閲覧日: 2026-03-04).
6. 37zigen. Union Find の計算量. 37zigenのHP. URL : <https://37zigen.com/union-find-complexity-1/>. (2026-06-11 現在リンク切れ).

本講座の執筆にあたって最も参考にしたのは、本アルゴリズムの計算量が $O(N + Q\alpha(N))$ であることがはじめて証明された文献 1 です。

文献 2 の証明も、コストの階層への分け方などが少し異なりますが、おおよそ 1 と同様の証明と言えると思います。なお、文献 5 の証明は、文献 2 の証明と同一です。

文献 3 は、分割統治法により計算量の漸化式を作り、分割する位置を調整することで計算量 $\alpha(N)$ を示すという形で書かれています。アイデアは理解しやすく感じましたが、(特に union by size の設定で) 記述量が増えてしまうと考えたため、本講座では採用しませんでした。

ただし、逆アッカーマン関数 $\alpha(N)$ の計算量を導出するためには (アッカーマン関数を持ち出す必要はなく)、直接逆アッカーマン関数を扱えばよいという考え方は本講座でも参考にしま

した。本講座の証明は，文献 1 の証明をこの考え方により細部の記述を書き換えたものといえます。

トップページ：[AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など：[Discord サーバー](#)



AtCoderは、世界最高峰の競技プログラミングサイトです。リアルタイムのオンラインコンテストで競い合うことや、5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#)

各サービス

[AtCoder](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[AtCoderCareerDesign](#)

AtCoder株式会社について

資料請求

[会社概要](#)

[AtCoder](#)

[FAQ](#)

[AtCoderJobs](#)

[プライバシーポリシー](#)

[利用規約](#)

© 2023 AtCoder, Inc. All rights reserved.